

Prahari

High-Level Design — cross-camera vehicle tracking across 26 departments, without centralising a single frame of video.

The atom is the sighting: about 200 bytes describing one vehicle at one camera. Video stays where it was recorded. The sighting is what crosses the department boundary.

ARCHITECTURE MODEL

Hybrid — M1 foundation, M2 surface, M3 core

SCALE TARGET

~50 heterogeneous cameras today, path to ~80,000

REPOSITORY

github.com/Priyanshu-byte-coder/prahari

DOCUMENT STATUS

Describes the system as built and measured, and names what is not

MANDATORY ANALYTIC

ANPR, with a four-reader vote and an explicit refusal band

LICENCE

Apache-2.0 · open source end to end, no vendor lock-in

DEMO VIDEO

youtu.be/8lQsuXeH150

- 1 What the system is
- 2 Integration approach
- 3 Storage
- 4 Watchlist correlation
- 5 Alert workflow
- 6 Delivery to the console
- 7 Route reconstruction

- 8 Security model
- 9 The privacy answer
- 10 Interop assumptions
- 11 Scale tiers
- 12 Failure modes
- 13 What is measured
- 14 What is not built yet

Status words in this document mean exactly what they say. **BUILT** is running and tested in the repository; **STUB** is the real protocol behind a labelled placeholder; **NOT BUILT** is designed and not written. An engineer who has never seen the repository should be able to rebuild the integration from this file alone.

1 What the system is

A vehicle passes a camera. Within a couple of seconds the plate is read, checked against a watchlist, and — if it matches — pushed to every console allowed to see that camera. Later, an investigator names that registration number and gets an ordered, timestamped path across the network, exportable and audited.

Twenty-six Gujarat government departments run independent CCTV: different vendors, analog and IP, separate VMS, separate storage, retention between seven and fifteen days. There is no unified asset inventory and no cross-department search. Prahari does not replace any of that. It federates it.

The design decision everything else follows from: **the product is the sighting, not the video**. A sighting is roughly 200 bytes — contract C1 — describing one vehicle seen at one camera over a short interval: plate text, confidence band, vehicle class, colour, bounding box, camera id, and the first and last frame presentation timestamps. Everything downstream reads sightings.

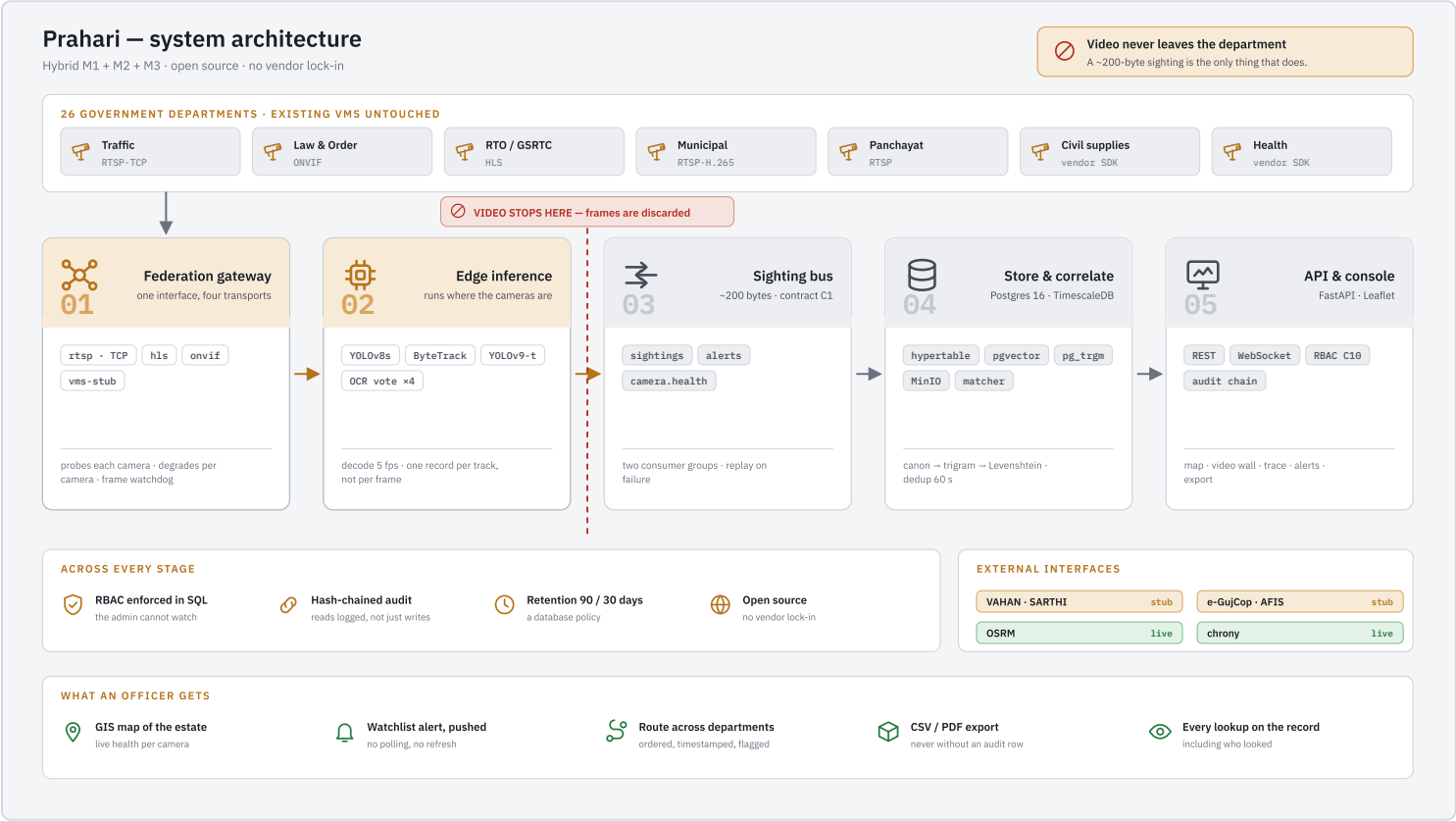


Figure 1 — System architecture. Frames are decoded where the camera already is and discarded. What crosses a department boundary is a ~200-byte row. The five stages map onto the reference models: the registry and GIS layer is M1, the single console surface is M2, and the federation gateway is M3. A full-page version is submitted separately as 03-prahari-architecture-diagram.pdf.

Why a hybrid rather than one of the four reference models

MODEL	WHAT IT GIVES	WHY IT IS NOT ENOUGH ALONE
M1 Centralised registry & GIS	One inventory, one map, department ownership per camera	Metadata only — it cannot answer where a vehicle went
M2 Unified viewing & analytics	One place for an officer to look	A viewing surface with nothing federating behind it is a bookmark folder
M3 VMS federation / middleware	Interop across vendors without centralising	No shared search unless something correlates across the seams
M4 Central VMS	Complete control	~160 Gbps and ~26 PB at fifteen-day retention for 80,000 cameras. It also requires replacing working departmental systems, which is a procurement, not an integration

We take M1 as the mandatory foundation, M3 as the core, and M2 as the surface. The bonus the brief offers for a hybrid is not why: the reason is that correlation is the requirement, and correlation needs a registry to key on, a federation layer to reach heterogeneous cameras, and one surface to show the result.

Component ownership

COMPONENT	CONTRACTS	STATUS
Federation gateway, transport probing, camera health, console	C6	BUILT
Decode, vehicle detection, tracking, plate localisation, OCR vote	C1	BUILT
Schema, persistence, matching, alerts, REST + WebSocket API, RBAC, audit	C3 C4 C5 C10	BUILT
VAHAN and e-GujCop feed clients	WatchlistFeed	STUB — awaiting agency credentials
Vendor VMS SDK driver	C6	STUB — interface complete

2 Integration approach

2.1 Cameras into the system

Cameras are not integrated one protocol at a time; they are integrated behind one interface.

CameraSource (contract C6) exposes open / frames / close / health / capabilities, and each transport is an implementation: MediaMTX-fronted RTSP, direct RTSP, HLS, and ONVIF where the camera supports events and PTZ.

Two field realities shaped this, and both were confirmed on the Sentinel grid:

- **RTSP over TCP, never UDP.** UDP dies across NAT and firewalls, and the failure presents as a frozen frame rather than an error.
- **Port 8554 is not reachable for every client.** On the sandbox the grid returns 401 on RTSP for our source IP, so the gateway probes each camera at startup and degrades that camera — only that camera

— to HLS, recording which transport actually worked in `camera:transport:<id>` in Redis.

The whole G→I seam is one Redis key holding a URL string. The inference worker imports no gateway code and the gateway knows nothing about models. That is what lets a new vendor arrive as one driver file rather than a release — and it is why the grid permitting exactly one of our four transports changed nothing downstream.

A stalled RTSP connection does not raise; it simply stops delivering. Health is therefore a watchdog on frame arrival rather than a `try/except`, and it publishes `camera.health` so the console can turn a pin amber within fifteen seconds instead of showing a frozen picture indefinitely.

2.2 Inference into the system

The worker consumes the transport URL, decodes at 5 fps behind a motion gate, detects vehicles with YOLOv8s, tracks them with ByteTrack, localises the plate with a trained YOLOv9-t ONNX detector (falling back to a classical blackhat/Sobel proposal), reads it with a four-engine vote, and publishes **one `C1` record per track** — not one per frame. A vehicle in shot for 200 frames is one sighting with a first and last timestamp, the best plate read across the track, and one crop as evidence.

Timestamps come from frame PTS, never `now()`. Arrival order across cameras is meaningless — one camera's frames may sit in a buffer while another's do not. A route assembled from arrival times reorders itself under load, which is precisely the failure a judge would notice. Every host runs chrony.

2.3 The seams

SEAM	SHAPE	DIRECTION
<code>camera:transport:<id></code>	JSON: url, transport, driver, probed_at	gateway → worker
stream <code>sightings</code>	<code>C1</code> record as one JSON field	worker → platform
REST + WebSocket	contracts <code>C4</code> , <code>C5</code>	platform → console
exchange files	<code>cameras.seed.json</code> , <code>camera_geo.json</code> , fixtures	one producer each

3 Storage

BUILT Postgres 16 with TimescaleDB, pgvector and pg_trgm (`db/schema.sql`, contract `C3`).

- `sightings` is a hypertable on `pts_first` with one-day chunks. The hypertable is created before the first insert — converting a populated table means a lock and an outage.
- Five indexes carry every read path: `plate_norm`, `plate_canon`, `camera_id`, a trigram index for fuzzy search, and an HNSW index on the 512-dimension re-ID vector.
- Compression after 7 days. Retention 90 days for sightings, 30 days for crops. **Alerts and audit rows are kept** — an alert nobody can look up a year later is worth little, and an audit chain with holes is not evidence.
- Crops live in MinIO over the S3 API, not in the database, and are reachable only through presigned URLs with a five-minute expiry issued after the scope check.

Measured: 14,991 synthetic sightings ingested at 50 rows/s for five minutes with zero consumer-lag growth on a laptop. Over the life of the project **100,366** sightings have gone through the live pipeline on one machine.

4 Watchlist correlation

BUILT `services/api/watchlist.py`, `feeds.py`, `matcher.py`.

4.1 Getting entries in

FEED	STATUS	NOTES
Manual entry	BUILT	what an operator types into the console
CSV import	BUILT	all-or-nothing, with a per-line error report
VAHAN	STUB	national vehicle registry: theft flag, blacklist status
e-GujCop (CCTNS)	STUB	FIR entities: wanted vehicles and persons

All four sit behind one protocol, `WatchlistFeed.pull(since)`. The two state-system feeds carry written-down request and response shapes and return sample rows labelled `STUB:<feed>`, so integration readiness is demonstrable without pretending the connection exists. Field names in those shapes are placeholders until the department issues API documentation and credentials — they are not a contract.

CSV import validates every row before writing any of them. A partial import is worse than a rejected one: the operator believes 200 vehicles are watched when 188 are, and nothing says which 12 are missing. A rejected row names its spreadsheet line number and a reason in words.

4.2 Matching a sighting to an entry

Every sighting is checked by a *second* consumer group on the same stream, so a slow match never delays a write.

- 1. Canon lookup, O(1).** `canon()` collapses every confusion class — `{0,0,D,Q}`, `{1,I,L}`, `{8,B}`, `{5,S}`, `{2,Z}`, `{6,G}` — so any single confusion-pair misread lands on the same key as the true plate. One dictionary hit, no scan.
- 2. Trigram fallback.** A plain misread changes the canon key, so a Postgres `similarity()` query at a 0.4 retrieval floor catches it. Measured: one edit in a ten-character plate scores 0.57, two edits 0.47, an unrelated plate 0.0.
- 3. Weighted Levenshtein decides the band.** A confusion-pair substitution costs 0.5; anything else costs 1.0.

COST	MEANING	BAND RAISED
0	exact string match, and the read itself was CONFIRMED	CONFIRMED
0	exact match on a less confident read	PROBABLE
0.5	one confusion-pair edit	PROBABLE
1.0 – 2.0	one plain edit, or two edits	POSSIBLE
> 2.0	a different vehicle	no alert

An alert is never more confident than the read behind it. A POSSIBLE read that happens to spell a watched plate does not put CONFIRMED in front of an officer. And only entries valid *now* are indexed: an expired entry that still fires produces an alert the operator has to dismiss, and a watchlist that cries wolf stops being read.

5 Alert workflow

BUILT `services/api/alerts.py`.

```
match → dedup check → NEW → ACKNOWLEDGED → ACTIONED
                                     ↳ DISMISSED # reason required
```

- **Dedup** per (watchlist entry, camera) for 60 seconds. A car at a red light is seen on twenty frames; twenty alerts is how an operator learns to ignore the panel. A repeat increments a count on the open alert. The window is per camera, so the same vehicle on the *next* camera is news.
- **The state machine is enforced server-side.** An illegal transition is HTTP 409 and writes nothing. `DISMISSED` requires a reason — dismissing without one is the failure mode that makes an audit log worthless.
- Every transition writes an `alert_events` row and an audit row, against a named user.
- The alert is published to a Redis stream, and the WebSocket fanout turns it into an `alert.new` frame for every console whose scope covers that camera.

Latency: the pipeline half — frame in to sighting published — is measured at **0.68 s** against a 3.0 s budget. Sighting to console is budgeted under 2 s on the RTSP path and under 6 s on HLS, where the floor is the segment duration rather than our code.

Prahari — workflow & integration

Two paths. Both end on the same hash-chained audit log.

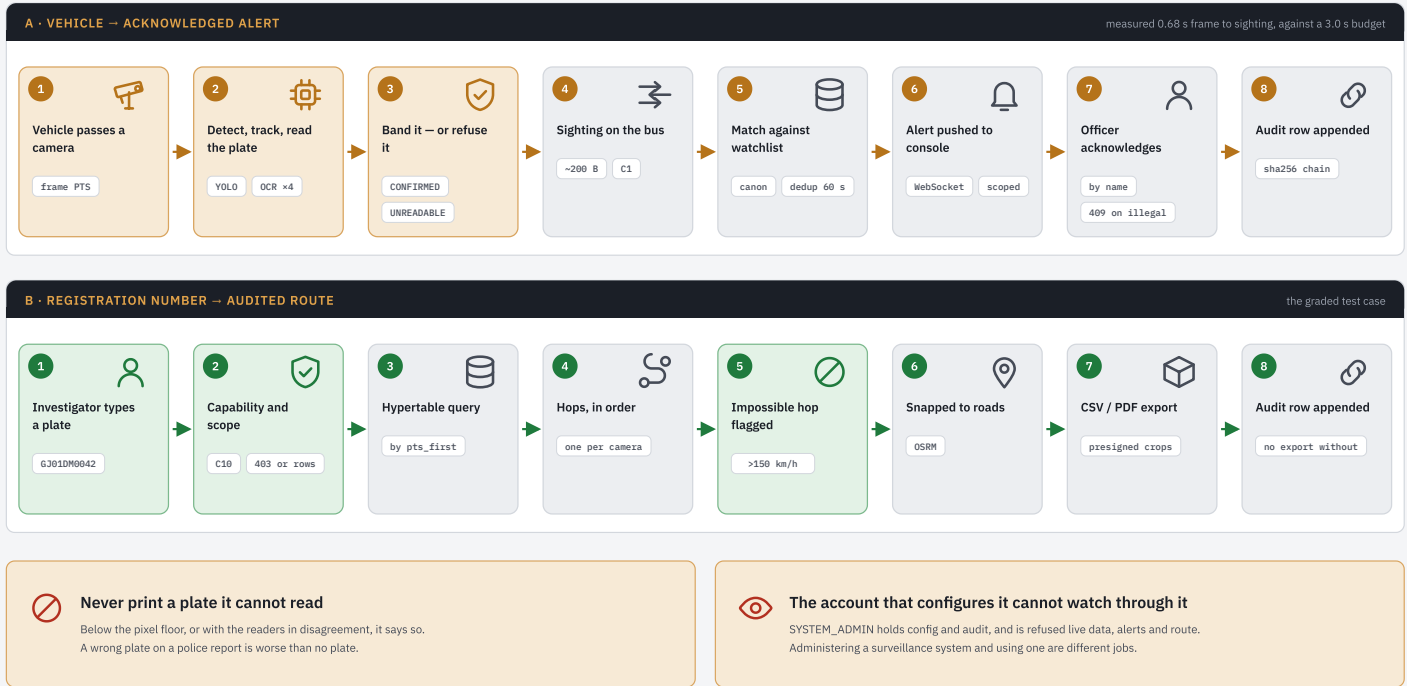


Figure 2 — Workflow and integration. Path A is the real-time route from a vehicle passing a camera to an alert acknowledged by a named officer. Path B is the graded test case: a registration number in, an audited route out. Both terminate on the same hash-chained audit log. *A full-page version is submitted separately as 04-prahari-workflow-integration-diagram.pdf.*

6 Delivery to the console

BUILT `services/api/ws.py`, contract C5.

One WebSocket per console replaces all polling. The client sends its JWT as the first frame; the server computes scope once and filters every frame server-side.

A console that receives frames it must not display and hides them in JavaScript has already leaked them. Scope is applied before the frame is on the wire, not after.

Frames carry a monotonic `seq`, and the last few thousand stay in a ring buffer. A console that drops off station wifi reconnects with `{"type": "resume", "since": N}` and receives exactly what it missed, filtered through its scope again. Without that, a reconnect is a blank panel and the operator cannot tell *nothing happened* from *I missed it*. Heartbeat every 15 seconds, because a dead TCP connection is indistinguishable from a quiet grid at 3 a.m.

Fanout reads the Redis streams with `XREAD` rather than a consumer group: a group splits messages between its members, so two API replicas would each show half the alerts to half the consoles.

7 Route reconstruction — the graded test case

BUILT `services/api/route.py`. A judge names a plate and gets its path.

The last row is the design, not an oversight: the most privileged account in the system cannot watch video. Administering a surveillance system and using one are different jobs, and the account that does both is the one an insider abuses. This is verifiable in the running system — signing in as `console-audit` returns 403 on alerts and route, and 200 on the audit endpoints.

Enforcement is three layers, in this order:

1. **The capability check** — the role either holds `watchlist:write` or gets a 403.
2. **The scope predicate in the SQL** — every read carries `(statewide OR owner_dept_id = ANY(:departments))`. Hiding a button is not access control.
3. **Postgres row-level security** — the backstop for a query somebody forgets to scope, keyed on session variables `(prahari.statewide, prahari.dept_ids)` set per request.

A user with no department matches nothing rather than everything: a misconfigured account should see an empty screen, not the state.

8.3 Audit

Append-only and hash-chained: `hash = sha256(prev_hash || canonical_json(fields))`. The chain does not *prevent* tampering — it makes tampering visible, which is the achievable property. Edit one row and every hash after it stops matching; `GET /api/admin/audit/verify` walks the chain and names the first break. *Verified: 728 rows, ok: true, first_broken_seq: null*.

Reads are logged, not just writes. In a surveillance system the abuse worth catching is *looking*: an officer checking an ex-partner's vehicle leaves no other trace. Exports, live views, watchlist changes and every alert transition are on one chain.

9 The privacy answer, in full

This is the question a citizen asks, and the answer is architectural rather than a policy promise:

- **No central video recording.** Video is decoded at the edge and discarded. What leaves a camera site is a ~200-byte row and, at most, a small crop of the vehicle. There is no archive to subpoena, leak or trawl, because there is no archive.
- **Retention is bounded and enforced by the database.** Sightings 90 days, crops 30 days, both as automatic policies rather than a cleanup script somebody remembers to run.
- **Access is purpose-bound.** A cross-department lookup requires a case number and a stated reason, both recorded.
- **Access is time-boxed.** Such a grant expires automatically, at most 72 hours, rather than living until somebody remembers to revoke it.
- **Access is scoped.** A department sees its own cameras. Statewide access is one role, held by investigators, and every use of it is logged.
- **Access is provable.** The hash-chained audit log means *nobody looked at this* is a claim that can be checked rather than asserted.

- **Crops are reached only through five-minute presigned URLs**, issued after the scope check. A leaked URL expires; a leaked bucket path is useless.

What the system deliberately does not do: face recognition against a general population, retention beyond the windows above, and any lookup that cannot name the case it belongs to.

10 Interop assumptions

Stated so an integrator can check them before writing code:

1. **Cameras** speak RTSP (TCP) or HLS. ONVIF is used where available for events and PTZ, and is not required. Mixed H.264 and H.265 in one grid is normal and handled.
2. **The grid catalogue** is read before decoding rather than probed blindly, and per-camera properties come from it. *Observed on the sandbox:* `/api/ingest` and `/live/stream/` return 404 on that deployment, and the grid permits **one session per source IP** — a second sign-in takes the first one's session. Both facts are handled in the gateway rather than assumed away.
3. **Clocks** are synchronised with chrony. Sighting order across cameras depends on it.
4. **VAHAN and e-GujCop** are reachable only with agency credentials issued by the respective department. Until those exist both feeds run as labelled stubs; swapping in the live client changes one class each and no calling code.
5. **Coordinates** come from a survey, not from district centroids. A centroid puts every camera in a district on one pixel and makes a traced vehicle teleport.
6. **Everything is open source.** Postgres, TimescaleDB, Redis, MinIO, OSRM, FastAPI, MediaMTX, Ultralytics, Leaflet. No managed service is required to run this — a contest rule, and also the difference between a pilot and a procurement.

11 Scale tiers

The deployment shape changes three times between a demo and a state rollout. The code does not.

The arithmetic that decides it

Centralised video — the M4 approach we reject

```
80,000 cameras x 2 Mbps = 160,000 Mbps
                        ~ 160 Gbps
160 Gbps x 86,400 s/day ~ 1.7 PB/day
at ~15-day retention   ~ 26 PB
```

Metadata — what we actually ship

```
~200 bytes per sighting row
80,000 cam x ~1,000/day ~ 80 M rows/day
80 M x 200 B           ~ 16 GB/day raw
Timescale compressed   ~ 2-3 GB/day
```

Tier 1 — one node (today, and the demo)

Everything on one machine: gateway, one or two workers, Redis, Postgres/Timescale, MinIO, API, console. Handles roughly 30 cameras at 5 fps analysis. The limit is GPU decode: H.264 decodes every frame even when analysis runs at 5 fps, so 30 cameras at 25 fps is ~750 fps of decode, near the ceiling of one

consumer-grade NVDEC. Watch `nvidia-smi dmon` ; above 90% decode utilisation, move cameras to CPU decode.

Tier 2 — a district (tens to a few hundred cameras)

Workers move to their own boxes, one per GPU, each pinned to a set of cameras from the registry. Redis stays central to the district. Postgres stays single-node; a Timescale hypertable on commodity hardware absorbs a few hundred sightings per second without trouble. The API scales horizontally behind a load balancer — the WebSocket fanout is broadcast-safe because every process reads the whole stream.

Tier 3 — statewide (thousands of cameras)

- **Edge workers** keep doing exactly what they do now, and buffer locally when the uplink drops.
- **Kafka replaces Redis Streams** as the trunk between districts and the centre. The migration is contained: the producer and the two consumer groups are the only code that knows which broker it is talking to, and the `c1` record does not change. Kafka is chosen for retention and replay across a WAN, which is where Redis Streams stops being the right tool.
- **Central Postgres/Timescale** with district-level partitioning, plus read replicas for the console.
- **Object storage** (MinIO or S3-compatible) for crops, with lifecycle rules enforcing the 30-day window.
- Re-ID vectors move to a dedicated index if HNSW inside Postgres stops keeping up.

The seams that make this possible already exist: a stream between inference and storage, a REST/WebSocket boundary between storage and the console, and no shared code across either.

12 Failure modes, and what the operator sees

The system is judged on a live grid, so this section is about what a person sees at 3 a.m., not about ideal behaviour.

FAILURE	DETECTION	WHAT THE OPERATOR SEES	RECOVERY
A district uplink drops	frame watchdog, no <code>camera.health</code> for 15 s	that district's pins turn red within 15 s, with a buffering badge	edge workers buffer locally and replay when the link returns; no gap in the timeline
A GPU or worker dies	its cameras stop producing sightings	affected pins go amber, then red	the registry reassigns those cameras to a surviving worker within 30 s
Postgres restarts	persister insert fails	nothing — the console keeps receiving live alerts	the consumer group has not acked, so the batch is redelivered; <code>ON CONFLICT DO NOTHING</code> absorbs the replay
A consumer dies mid-batch	its messages stay pending	nothing	<code>XAUTOCLAIM</code> moves anything idle past the threshold to a live consumer; without this a killed worker is a permanent hole
RTSP refused for a camera	transport probe fails at startup	that camera shows an HLS badge	the gateway degrades that camera to HLS; latency rises to the segment duration
A console loses connectivity	socket closes	reconnect banner	it reconnects with <code>resume: seq</code> and the server backfills exactly the gap
OSRM is down	request times out in 3 s	the route still draws, as straight lines, labelled unsnapped	none needed; the hops, times and flags are the evidence
MinIO is unreachable	presign raises	hops render without thumbnails	the route and the alert are unaffected
A plate is misread into a watched plate	plausibility check on the route	the hop is shown, flagged <code>IMPLAUSIBLE</code> , with the implied speed	the operator sees the doubt instead of a confident lie

13 What is measured

Every number in this document comes from a command in the repository, and every one of them can be re-run by a judge.

COMMAND	RESULT
<code>pytest tests/ -q</code>	372 passed, 6 skipped
<code>python scripts/verify_stack.py</code>	35 / 35 checks passed against the running stack
<code>python services/worker/selftest.py</code>	SELFTEST OK — plate read back exactly, 0.68 s against a 3.0 s budget
<code>PRAHARI_INTEGRATION=1 pytest tests/test_integration.py</code>	6 / 6 — clip → worker → real Redis stream → persister → matcher → alert → route
<code>python scripts/accuracy_report.py</code>	95.0% exact on 40 tracks, 0 confidently-wrong reads , 5.0% refused
<code>GET /api/admin/audit/verify</code>	728 rows, ok: true
<code>python scripts/preflight.py</code>	13 / 13 , and every failure prints the command that repairs it

ANPR accuracy, banded

BAND	N	SHARE	WRONG	READER ALONE	EXACT	CER
CONFIRMED	38	95.0%	0	paddleocr	79.2%	0.121
PROBABLE	0	0.0%	0	easyocr	70.0%	0.221
POSSIBLE	2	5.0%	0	fast-plate-ocr	55.0%	0.241
NONE	0	0.0%	0	tesseract	42.5%	0.257
				fused vote	95.0%	0.050

Provenance: this set is synthetic, generated by `scripts/accuracy_report.py` on 2026-09-04, and the document says so rather than rounding it into a headline. A hand-labelled Gujarat set is open work.

Low-resolution plates — the case the grid actually presents

Wide-area cameras on the Sentinel grid put a plate into 20–30 px of frame. The published reference for this regime (UFPR-SR-Plates, median plate height 18–21 px) reports **1.7–2.2%** recognition from a low-resolution crop straight to OCR, rising to **42.3–44.7%** with super-resolution plus majority vote by character position. Our measurement on the same regime:

PLATE HEIGHT (NIGHT)	LR → OCR	SINGLE-IMAGE SR	OURS — MFSR ENSEMBLE + MVCP
16 px	0.0%	0.0%	5.0%
20 px	0.0%	0.0%	65.0%
24 px	0.0%	15.0%	95.0%

There is a cliff just under 20 px, and the system is built around admitting it. A plate face is roughly 65% glyph, so 20 px of plate is about 13 px of glyph — right at the bottom of what CRNN and SVTR recognisers were trained on. `preprocess.GLYPH_MARGINAL_PX` is set from this measurement: below about 10 px of glyph the pipeline refuses and reports *plate not resolvable*, and the video wall says so on the tile rather than printing four confident characters of noise.

14 What is not built yet

Named plainly, because a design document that hides its gaps is not useful:

- **Re-ID corroboration of route hops** — the route ships plate-keyed; a hop supported by appearance similarity is a planned upgrade, and the vector column and index already exist.
- **Cross-department grants** — the schema and the audit fields exist; the request and approval flow is next.
- **Live VAHAN and e-GujCop connections** — blocked on agency credentials, stubbed behind the real protocol.
- **Fine-tuned detection and TensorRT export** — pretrained models ship first; a detector fine-tuned on labelled Gujarat footage is the single biggest accuracy lever remaining.
- **A hand-labelled accuracy set** — the banded numbers above are honest about being synthetic.
- **Analytics beyond ANPR** — crowd counting, wrong-way and loitering detection are designed against the same `c1` record and not yet written.

Two rules run through every decision in this document. Never print a plate the system cannot read — a wrong plate on a police report is worse than no plate. And the account that configures the system cannot watch through it — because administering a surveillance system and using one are different jobs.